

Case Study

The goal of this task is to build an application/platform to manage job listings.

General

- Take all the time you need. Pause/resume your work as you like.
- You can use all online resources, including search engines and documentations, to complete the test. If you have structural questions, please don't hesitate to ask us.
- All code must be written by yourself. Don't copy & paste from external sources or tutorials. Don't "outsource" this task to someone else. This test does not only give us an impression about your skills, it also gives you an impression about what you might face working with us.

Tech Stack

- Must have
 - o Backend: Laravel, PHP
- Nice to have
 - o Frontend
 - CSS: Use one of Zurb Foundation, Tailwind or Twitter Bootstrap 4. (Compile/Generate the CSS (only SCSS) and JavaScript yourself, don't use the default CSS.)
 - JS: VueJS (Prevent using other JavaScript frameworks and libraries (including JQuery))

Features

We focus on the backend and API part. All frontend/view tasks are optional. It looks basically like that

- The platform **has many Jobs**
 - o Model: Job.php
 - o (Resource-) Controller: JobController.php
 - o Policy: JobPolicy.php
 - o Repository if needed: JobRepository.php

- Each job **belongs to** a **company**
 - o Model: Company.php
 - o (Resource-) Controller: JobController.php
 - o Policy: CompanyPolicy.php
 - o Repository if needed: CompanyRepository.php
 - o Each company **has many jobs**
- Each Company **has many users**
 - o Model: User.php
 - o (Resource-) Controller: UserController.php
 - o Policy: UserPolicy.php
 - o Repository if needed: UserRepository.php
 - o Each user **has many companies**
- Must have
 - o Basic model structure as above. If needed, add additional models, pivots or other classes as needed.
 - o API-Endpoints to handle each model of your application (<https://laravel.com/docs/5.8/controllers>)
 - Create (POST), i.e. localhost/jobs
 - Update (PUT), i.e. localhost/jobs/{id}
 - Index (GET), i.e. localhost/jobs
 - Show (GET), i.e. localhost/jobs/{id}
 - Some for companies, users, categories etc.
 - o Every request follows this pattern (simplified)
 - Request some url (i. e. POST localhost/jobs)
 - Find route: routes.php/api.php/web.php
 - Forwarded to correct controller method (i.e. JobController@store)
 - Run Policy in your Policy Class
 - (i. e. JobPolicy.php, \$user->can('store'...) or \$this->authorize('create', Job::class);
 - Validate data (\$this->validate(\$request, ['title' => 'string|255']))
 - Run business logic in your repository (i.e. JobRepository.php)
 - Give back response based on the events happened (return response(\$originator);)
- Optional/Nice to have/suggestions
 - o Add a frontend with the following features

- CREATE jobs from
- CREATE companies form
- SHOW jobs detail page
- SHOW companies detail page
- Dashboard with relevant information
- TopBar Navigation
- Footer
- (Nested) categories
- Image Upload for Company Logos and Jobs
 - Transform images into different sizes automatically during the upload
- Admin Backend
 - Simple user interface to manage models (i. e. Laravel Nova)
- Cache us much as possible
- Bind Models to routes
- Unit Tests for automatic testing
- Breadcrumbs
- Add features on your own (Big plus)

Notes

- General
 - THINK (!!!) before you act. Don't just do in panic what someone told you, understand the purpose and do whatever you do for a reason! If you need time to question a task, that's fine! To deliver a mediocre result is not. What models are you going to create, what properties will they have? How and why did you chose those values/properties? What relationships will they have? How can you improve your design? Are there best practices to consider? Decisions must always be taken on a valid and stable basis, which you can explain at any time.
 - **We never want any bad example given as an excuse. Instead, we look at the best and try to improve from there.**
 - Use artisan, it automates and standardizes big chunks of the routine development tasks, i. e.
 - `artisan make:model Job -mcr` give you a perfectly prepared Controller, the Model and a migration file in one command.
 - `artisan make:policy JobPolicy --model:Job` gives you a Policy Stub
 - Use Laravels shortcuts, i.e.

- Put `Route::resource(jobs, 'JobController');` into your routes file to have all rest routes defined and linked with the corresponding controller instead of defining them all by yourself.
 - Don't repeat yourself (DRY, https://en.wikipedia.org/wiki/Don%27t_repeat_yourself)
 - If you feel like you are writing and using the same piece of code, or minor variations of it, multiple times, put it into a central place where you can reuse it.
 - Separation of concerns (https://en.wikipedia.org/wiki/Separation_of_concerns)
 - Please keep each file as slim as possible and separate everything into the right place (Routes, Controllers, Repositories, Models) instead of writing super long methods.
 - Please keep in mind that we need to understand what and how you have done it.
 - We must be able to fully rebuild the project locally, based on your GitHub repository, including the database.
 - Use seeder classes to populate your database if needed. Don't add anything to your DB manually, otherwise we can't run/build your code.
 - Please make sure that you name all files, classes, functions properly so that an external developer can follow your steps.
 - Instead of committing all changes at the same time, try to commit and push your features one by one to GitHub. This will allow us to get a better impression of what you did. (For example, "Added resource controller for jobs", "Added frontend edit view for categories", "Added filters to search view" instead of "Everything I did the last 17 days")
 - You get credit for every line you DON'T write
 - Keep your code as simple, slim and lightweight as possible. Complicated code is not a good thing, it's bad! Code nobody understands, also bad.
 - Be consistent on how you name variables, functions and classes
- Notes about Models
- You will need many models very quickly as every model as a limited scope. (Elloquent, i. e. User, Job, Category, Tag, JobTag, CategoryTag, Menu, MenuItem, Company, CompanyJob etc.) To keep it simple, start lean and expand as you go.
 - Don't forget to define all model properties and validation rules correctly

- Don't forget to define all possible relationships (Important)
- Notes about Controllers
 - All routes should follow proper RESTful design
 - Have at least two resource controllers with all CRUD-operations working.
 - Use i.e.

```
artisan make:controller JobController --resource
```

 or

```
artisan make:model Job -mcr
```


to get all methods stubbed
 - Don't forget to properly authenticate and authorize all requests and validate all posted input data for create and edit requests (use Policies). Everyone can see jobs, only authenticated users can create posts, only the creator of a post is allowed to edit it.
- Notes about Views (optional)
 - You will need your layout views (master), holding the navbar and content containers.
 - You will need at least 3 different views for every model, (Edit/Create, List, Detail)
 - Separate your views in smaller and reusable chunks, i.e., one master view which has different child views
 - The header must have only one minified CSS and JS file.
 - Don't write any language strings directly into your blade templates. Always use language files (i.e. `/resources/lang/en/blog.php` and `{{ trans('blog.test') }}`)

First steps

- Most important: Take some time to think about this project and create a small concept for yourself. Add your notes inside the project readme.md file.
- Create a new, clean Laravel project.
- Upload your (clean) project to GitHub before you get started with your development work. Make it private if you want. Don't forget to commit your readme.md.
- Start building your application with all the must haves we defined.
 - i. Start with the backend logic
 - 1. Create all Classes, including Models, Migrations, Controllers and Repositories
 - 2. Get to the point, where you can manage all models as described
 - ii. Add a frontend, if you have time

- Extend the application as you like
- Commit everything and push your changes to GitHub